Project Final Report
Implementation and Verification of the Union-Find Data Structure in Stainless

Mathilde Peruzzo
mathilde.peruzzo@epfl.ch

Rassene M'sadaa
rassene.msadaa@epfl.ch

Julian Marmier
julian.marmier@epfl.ch

October 19, 2026

Abstract

The Union-Find data structure is fundamental to efficient graph algorithms, yet its reliance on mutable state and pointer manipulation poses significant challenges for formal verification. This project presents a formally verified implementation of a Disjoint-Set Forest in Stainless. We adopt a flat-memory model using functional lists to simulate the heap, enabling the verification of functional correctness and termination without relying on imperative pointers. We formalize the structural invariants derived from the work of Charguéraud and Pottier, specifically using rank monotonicity to prove the absence of cycles and the termination of recursive operations. We demonstrate that Stainless can effectively verify complex heap-like data structures through modular proof strategies and extensive `List` lemmas.

1 Introduction

The Union-Find data structure is a cornerstone of efficient graph algorithms, such as Kruskal's algorithm for Minimum Spanning Trees. While its imperative implementation is well-understood, its formal verification is non-trivial due to the reliance on mutable state, pointer aliasing, and the potential for cyclic structures. In functional verification environments like Stainless [3], which rely on pure mathematical functions, verifying such structures requires bridging the gap between imperative performance and mathematical purity.

The primary challenge lies in modeling a mutable heap within a functional setting while allowing the solver to reason about state changes and structural invariants. Specifically, that the structure remains acyclic and that recursive operations terminate. This project presents a mechanically verified implementation of Disjoint-Set Forests in Stainless that addresses these challenges.

Our work adapts the separation logic concepts established by Charguéraud and Pottier [1] into a first-order functional setting. Specifically, we've achieved:

1. **Functional Memory Model.** We simulate a mutable heap using an immutable `List` structure and index-based addressing. This effectively models the heap without the complexity of pointer logic, allowing us to reason about "pointers" as simple integer indices.
2. **Rank-Based Termination.** We enforce a strict rank-monotonicity invariant (a parent's rank is strictly greater than its child's). We use this invariant to prove acyclicity, guaranteeing that the recursive **Find** operation always terminates without infinite loops.
3. **Auxiliary Proof Library.** We developed `MoreListSpecs`, a specialized library of verified lemmas. This library handles the complex properties of `List` operations such as preserving invariants across `updated` and `append`, decoupling the data structure logic from the underlying storage proofs.

2 Theoretical Background & Specification

This project relies on the formalization strategies presented by Charguéraud and Pottier in *Machine-Checked Verification of the Correctness and Amortized Complexity of an Efficient Union-Find Implementation* [1]. In their work, they verify an imperative OCaml implementation using Separation

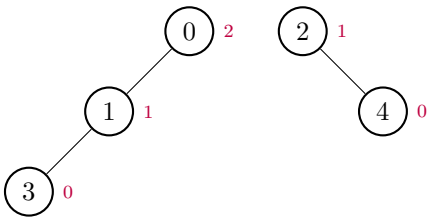


Figure 1: Example diagram of a disjoint-set forest over domain of integers between 0 and 4 inclusive. The domain is partitioned as $\{0, 1, 3\}$ and $\{2, 4\}$. Each node's *rank* is shown on the side.

Logic. Here, we review the key theoretical concepts and predicates that define the correctness of the data structure, which serve as the "ground truth" for our Stainless implementation.

2.1 Disjoint-Set Forests

A disjoint-set forest is a data structure used to represent a partition of a set of elements D (the domain). It is modeled as a directed graph where every node $x \in D$ has at most one outgoing edge to a parent node.

The structure is defined by an edge relation F, where $F(x) = y$ indicates a directed edge from x to y.

- **Roots:** A node r is a root if it has no successors in F.
- **Representatives:** A node x is represented by a root r if there is a path from x to r.
- **Validity:** A graph is a valid disjoint-set forest if F is functional (nodes have at most one parent) and acyclic (every node deterministically reaches a root).

Each node has a rank k which for our purposes we define as the number of levels beneath the node. For instance in the example shown in Figure 1, node 0 has rank 2, nodes 1 and 2 have rank 1, and nodes 3 and 4 have rank 0.

2.2 The Charguéraud-Pottier Formalism

To bridge the gap between abstract sets and the concrete memory layout, the original paper utilizes three core predicates.

The Invariant Predicate (`Inv`). The predicate $\texttt{Inv}(N, D, R, F, K)$ asserts that the abstract structure represents a valid **Ranked Disjoint-Set Forest**. It ensures that:

1. The domain D has cardinality bounded by N.
2. The mapping R (element to representative) is consistent with the graph structure defined by F.
3. The ranks K satisfy the **Rank Monotonicity** property: for any edge $x \to y$, $\text{rank}(x) < \text{rank}(y)$.

The Memory Predicate (`Mem`). The predicate $\texttt{Mem}(D, F, K, M)$ relates the abstract model to the physical memory heap M. It asserts that for every node $x \in D$:

- If x is a child pointing to y in the model ($F\,x\,y$), then the memory at address x contains a pointer to y.
- If x is a root with rank k in the model, then the memory at address x stores the value `Root` and rank k.

The Representation Predicate (`UF`). The high-level predicate $\texttt{UF}(N, D, R)$ combines the above. It states that there exists a physical heap M and an abstract forest (F, K) such that both `Inv` and `Mem` hold. This separates the client's view (sets and representatives) from the internal implementation details.

2.3 Union by Rank and Acyclicity

To ensure efficiency and guarantee acyclicity, the algorithm employs the **Union by Rank** heuristic. The critical invariant maintained by the data structure is that the rank of a parent node must be strictly greater than the rank of any of its children.

$$\forall x, y.\, F(x) = y \implies K(x) < K(y) \tag{1}$$

Because ranks are natural numbers and strictly increase along edges, it is impossible to form a cycle. This invariant provides a strictly decreasing metric for the recursive **Find** operation, ensuring its termination.

2.4 Specification of Operations

We aim to verify the standard operations of a Disjoint-Set Forest. Based on the formal model from [1], the specifications are:

1. **Make**(x): Adds a new element x to the domain.
   - *Precondition:* $x \notin D$.
   - *Postcondition:* The new structure contains $\{x\}$ as a singleton set, where x is a root with rank 0.
2. **Find**(x): Locates the representative of the set containing x.
   - *Precondition:* $x \in D$.
   - *Postcondition:* Returns r such that r is a root and x is connected to r. The graph structure remains semantically equivalent.
3. **Link**(r_1, r_2): Merges two trees rooted at r_1 and r_2.
   - *Precondition:* r_1 and r_2 are roots.
   - *Postcondition:* The root with the smaller rank becomes a child of the other. If ranks are equal, r_1 becomes a child of r_2, and $\text{rank}(r_2)$ is incremented by 1.
4. **Union**(x, y): Merges the sets containing x and y.
   - *Postcondition:* Calls **Find**$(x)$and **Find**(y) to obtain roots, then calls **Link** on them.

3 Implementation in Stainless

The primary challenge of this project was translating the separation logic model, which relies on mutable heaps and pointer arithmetic, into the strictly functional and immutable environment of Stainless. Our implementation replaces the physical heap with a functional memory model while maintaining the rigorous invariants defined in Section 2. It is available on GitHub at math-peru/union-find.

3.1 The Functional Memory Model

In the original specification, the predicate $\texttt{Mem}(D, F, K, M)$ relates abstract sets to mutable memory cells. In Stainless, we simulate this memory M using an immutable `List`:

```
type Heap[T] = List[Node[T]]
```

In this model:

- **Addresses** are represented as `BigInt` indices into the `List`.
- **Pointers** are simply integer fields within a node that store the index of another node in the same `List`.
- **Updates** are performed functionally using `List.updated(addr, newNode)`, which returns a new `List` rather than modifying the existing one in place.

This design choice eliminates the need for complex heap predicates but requires us to manually maintain structural validity (e.g., ensuring indices remain within bounds after `List` operations).

```
sealed trait Node[T] {
  val addr: BigInt
  val rank: BigInt
  val value: T
}

case class Child[T](addr: BigInt, value: T, rank: BigInt, parentAddr: BigInt)
    extends Node[T]

case class Root[T](addr: BigInt, value: T, rank: BigInt)
    extends Node[T]
```

Figure 2: The `Node[T]` ADT which make up the building blocks of our implementation.

3.2 Node Abstraction

Figure 2 describes the `Node[T]` algebraic data type to distinguish between roots and children explicitly. This distinction simplifies the implementation of operations like **Link** and **Find**, allowing pattern matching to replace explicit checks.

3.3 Formalizing Invariants

We translated the global `Inv` predicate into a set of executable Scala invariants (enforced via `require`). These invariants are checked statically by the Stainless solver.

3.3.1 Structural Integrity (Invariants I, II, III)

These invariants ensure the consistency of our "flat memory" model, replacing the spatial component of separation logic.

- **Invariant I (Pointer Validity):** Every `parentAddr` stored in a `Child` node must be a valid index within the current heap bounds ($0 \leqslant addr < size$).
- **Invariant II (Self-Consistency):** Every node stored at index i must satisfy `node.addr == i`.
- **Invariant III (Domain Integrity):** The List of addresses must exactly equal the range $[0, size[$.

Note that invariant II is a consequence of invariant III.

3.3.2 Topological Validity (Invariants IV, V)

These invariants enforce the acyclicity and rank properties required for termination.

- **Invariant IV (Rank Monotonicity):** A parent's rank must be strictly greater than its child's rank.

$$\forall n \in \texttt{Heap}, n \text{ is Child} \implies \texttt{rank}(n) < \texttt{rank}(\texttt{parent}(n)) \tag{IV}$$

  This is the direct translation of the acyclicity requirement. It provides the decreasing metric for recursive functions.

- **Invariant V (Rank Boundedness):** The rank of any node is bounded by the number of children in the heap.

$$\forall n \in \texttt{Heap}, \texttt{rank}(n) \leqslant \{n : n \in \texttt{Heap} \wedge n \text{ is a } \texttt{Child}\}.\texttt{size} \tag{V}$$

  A corollary for this is that $\forall n \in \texttt{Heap}, \texttt{rank}(n) \leq \texttt{Heap.size}$. This assists the solver in proving that ranks do not grow indefinitely.

3.4 Usage of Large Language Models

The arrival of more intricate Large Language Models such as OpenAI's ChatGPT 5.2 [4] and Google's Gemini 3 [2] proved quite useful during the implementation of the Union-Find. LLMs particularly excelled at proving simpler lemmas such as the recursive list proofs in `MoreListSpecs.scala`, as well as generating documentation for our code. However they struggled in proving less trivial lemmas and proofs, notably those in `UnionFindList.scala`, `InvariantHelpers.scala` and `UnionFindSpecs.scala`, for which we did most of the work.

4 Verification of Operations

In this section, we detail the verification of the Union-Find data structure. We first demonstrate how each core operation maintains the structural invariants of the heap (safety), explaining the specific lemmas and logical steps required to satisfy the SMT solver. We then present the high-level functional specifications that define the correct behavior of the data structure (correctness).

4.1 The Verification Strategy

Proving that these invariants are preserved across operations, especially **Make** (which appends to the `List`) and **Link** (which updates an element), proved non-trivial. The SMT solver often struggles to infer properties of updated lists automatically.

To solve this, we implemented a modular proof strategy:

1. **MoreListSpecs:** We isolated generic `List` lemmas (e.g., *"updating an element at index i does not change the value at index j"*) into a separate library. These are detailed in Appendix A.
2. **Invariant Preservation Lemmas:** For each operation, we wrote specific helper lemmas proving that if an invariant holds for heap H, it holds for $H' = H.\texttt{updated}(\dots)$.

This decomposition allowed us to verify the functional operations producing a new data structure (**Make**, **Link**, and **Union**).

4.2 Preservation of Structural Invariants

The primary verification challenge is proving that operations which modify the heap, specifically updates and appends, preserve the global invariants defined in Section 3. Since Stainless strictly enforces invariants at every step, we must prove that the heap remains valid after every single modification.

4.2.1 Atomic Updates (`set`)

The `set` operation is the internal primitive used to update a node at a specific address. Since replacing a node effectively creates a new `List`, the solver effectively "forgets" properties held by the old `List`. We must strictly prove that the new heap H' satisfies all invariants $I(H')$. We achieve this by explicitly invoking preservation lemmas immediately after the update:

- **Structural Integrity:** We invoke `MoreListSpecs.mapUpdate` to prove that updating a node's content does not change its address field, ensuring the domain of addresses remains strictly $0 \dots size - 1$.

  This implies *self-consistency*, which is proven in the lemma `rangeInvImpliesAddrInv`.
- **Rank Monotonicity:** We invoke `rankInvUpdate`, a recursive lemma from `InvariantsHelpers`. This lemma iterates over the entire heap to prove that if the new node n satisfies local rank constraints (e.g., $rank(n) < rank(parent)$) and the old heap was valid, then the property holds for all nodes in the new heap.

  Rank boundedness is proven similarly.
- **Pointer Validity:** We use `parentInvUpdate` to prove that updating a node does not invalidate the parent pointers of other nodes. This is critical because a standard list update could theoretically change indices; we must prove it preserves the specific integer values used as pointers.

4.2.2 Extending the Domain (Make)

The **Make** operation creates a singleton set by appending a new root to the heap. The challenge here is the "Frame Problem" for appends: proving that adding an element at index N does not invalidate invariants at indices $0 \dots N - 1$. We verify this using a "divide and conquer" strategy with append-specific lemmas found in `InvariantHelpers`:

- **Structural integrity:** We invoke `MoreListSpecs.rangeAppend` to increase the range and `MoreListSpecs.mapDistributesOverAppend` to prove that mapping over the increased heap results in the increased range.
- **Self-consistency:** We use `addrInvAppend` to prove that existing nodes retain their correct addresses even after the `List` grows. Note that we could also have used `rangeInvImpliesAddrInv` here.
- **Rank Monotonicity:** We prove the implication $(\forall x \in H, I(x)(H)) \wedge I(new)(H) \implies \forall x \in (H + new), I(x)(H + new)$ using `rankInvAppend`, where I is the invariant function asserting rank monotonicity (IV). Since the new node is a root with rank 0, it trivially satisfies the invariant locally. The lemma then propagates this validity to the entire extended heap.

  Pointer validity and rank boundedness are proven similarly.

4.2.3 Termination of Path Traversal (FIND)

The **FIND** operation recursively traverses parent pointers. To prove termination in Stainless, we must define a measure that strictly decreases at each step. We utilized the **Rank Monotonicity** invariant to construct this metric. Since a parent's rank is strictly greater than its child's ($rank(child) < rank(parent)$), we defined the measure as:

$$\mu(n) = \text{Heap.size} - \text{rank}(n)$$

To validate this measure, we explicitly guide the solver using manual instantiation:

1. We assert that the current node n is in the heap.
2. We invoke `ListSpecs.forallContained` to instantiate the global rank invariant specifically for n.
3. This forces the solver to recognize that $parent.rank > n.rank$, which mathematically implies $\mu(parent) < \mu(n)$.

This guarantees that cycles are impossible and that the recursion is finite.

4.2.4 Merging Sets (LINK) and Simultaneous Updates

The **LINK** operation merges two trees rooted at *addresses* r_1 and r_2. The verification depends on the rank comparison:

- **Case** $rank(r_1) < rank(r_2)$**:** We update r_1 to be a child of r_2. This involves a single call to `set`. It trivially preserves rank monotonicity because the new edge points to a strictly higher-ranked node.
- **Case** $rank(r_1) = rank(r_2)$**:** This scenario is complex because it requires **two** updates: r_1 becomes a child of r_2, and r_2 must be updated to become a root with rank $rank(r_2) + 1$.

Updating the heap sequentially risks creating an invalid intermediate state (e.g., where ranks are temporarily equal, violating monotonicity). To solve this, we implemented a specialized private helper `setTwo`. Inside `setTwo`, we verify that the order of updates is commutative using `MoreListSpecs.updateOrderDoesNotMatter`. We specifically prove that converting a Root to a Child strictly increases the count of non-root nodes in the heap (via `updatedFilterSizeIncreases2`). This provides the necessary "budget" to safely increment the new parent's rank without violating Invariant V (Rank Boundedness).

4.3 Functional Specification and Verification

Beyond structural safety, we verified that the operations adhere to the high-level semantics of a Disjoint-Set Forest. These specifications are defined in `UnionFindSpecs`.

4.3.1 Basic Correctness

We formally specified and proved the following post-conditions for the core operations:

- `makeReturnsASingletonSet`**:** When creating a new element, the resulting node is guaranteed to be a root, point to itself (if found), and have a rank of 0.

- `findReturnsRoot`: For any valid address, **Find** ensures the returned node satisfies the `isRoot` predicate. This guarantees that the traversal always reaches a valid representative.
- `linkReturnsARootOfInput`: The **Link** operation guarantees that the returned representative is strictly one of the two input roots (`res ==` $a_1 \vee$ `res ==` a_2), ensuring that no new or arbitrary roots are introduced during a merge.

4.3.2 Compositional Verification of Union

The **Union** operation is verified by composing the contracts of **Find** and **Link**.

$$\textbf{Union}(a, b) \text{ ensures } \texttt{res} \in \{\textbf{Find}(a), \textbf{Find}(b)\}$$

Because we have proven that **Find** returns valid roots and **Link** correctly merges valid roots, Stainless automatically verifies that **Union** maintains the disjoint-set properties without requiring new inductive proofs.

4.3.3 Set Membership Properties

We also defined a more complex specification, `unionMergedTheSets`, stating that any element belonging to the set of a or b before the union belongs to the merged set afterward.

$$\begin{aligned} \forall x, ((\textbf{Equiv}(x, a) \vee \textbf{Equiv}(x, b)) &\implies \textbf{Find}'(x) = \textbf{Union}(a, b)) \\ \wedge((!\textbf{Equiv}(x, a) \wedge !\textbf{Equiv}(x, b)) &\implies \textbf{Find}'(x) \neq \textbf{Union}(a, b)) \end{aligned}$$

Verifying this property requires quantifying over the entire domain for every update. While we provided the specification, proving it fully for every case proved to be a significant challenge for the SMT solver, requiring extensive manual instantiation of (unproved) lemmas found in the `mergeset` branch.

5 Results and Challenges

5.1 Verification Results

Our verified implementation represents a significant effort, bridging the gap between high-level graph theory and low-level index manipulation. The scale of the project is summarized by the following metrics:

- **Verification Conditions:** The project generates over 900 proof obligations. These cover array bounds checks, pattern matching exhaustiveness, and the preservation of complex invariants across recursion.
- **Success Rate:** The majority of these conditions are discharged automatically by the Stainless solver, validating the correctness of our definitions.

  While safety and termination are fully verified, the functional property `unionMergedTheSets` is partially proven. Verifying the quantification over the entire domain for every update remains a challenge for the SMT solver within the current proof architecture. A partial proof exists on the branch `mergeset`, which attempts to prove that for any b represented by address $a_1 = \textbf{Find}(b)$, linking a_1 to a_2 implies either $\textbf{Find}(b) = a_1$ if a_1 absorbs a_2, or otherwise $\textbf{Find}(b) = a_2$ by examining the parent chain of b, which is constructed by running $\textbf{Find}(b)$ and recursively appending its parents such that the representative is at the head of the list. By checking that the parent chain before linking is a sublist of the parent chain afterwards (with enough well-chosen list specs) it could be concluded that $\textbf{Find}(b) = a_1$. However due to time constraints we were unable to fully finishing proving this.
- **Path Compression:** While we were unable to prove the validity of the **Find** operation when using path compression for optimization, we did define this operation and start on its verification. The basic idea is to reroute each child to the found representative while going back down the parent chain in **Find**. The challenge is to prove that the preconditions of `set` are satisfied at each recursive execution. To mitigate this, we decided to try to make all updates at once, after the recursive calls are finished and the representative is found. We defined a helper, `rerouteMultiple`, that updates the heap multiple times and tries to prove

the invariants on that modified heap before creating a new Union-Find. However, due to time constraints, we were unable to finish verifying this operation. Our current progress in available in branch `pathcomp`.

- **Code Structure:** We structured the solution into three distinct modules: the core algorithm (`UnionFindList`), the specifications of the methods (`UnionFindSpecs` 7 specs), the invariant preservation proofs (`InvariantHelpers` 5 invariant definition + 10 helpers for invariant proving), and the generic `List` library (`MoreListSpecs` 26 new lemmas). This separation allowed greater code clarity and solver performance.

5.2 Evolution of the Approach

Arriving at the final verified solution required a long path of testing different implementations and proof strategies.

Implementation choice. Our initial approach attempted to model the Union-Find structure using `Map` data structures to closely mimic the mathematical definition of a graph (D, F). However, reasoning about state changes in a map (where keys can be added or removed) is difficult as Stainless offers less support for maps as it does for lists.The breakthrough came when we pivoted to the **Flat Memory Model** described in Section 3. By representing the heap as a simple `List[Node]` where "pointers" are just indices, we eliminated the need to prove that keys exist in the map, and reduced "graph validity" to simple integer bounds checks ($0 \leq i < size$). This simplification drastically reduced the complexity of the proofs.

Proving Termination. Proving that **Find** terminates was difficult because the path length is dynamic. Early iterations tried to either (a) decrease the "distance to root," but this distance changes during linking (and path compression), making the invariant hard to preserve, or (b) starting a counter at `Heap.size` and decreasing it and asserting that it never reaches 0, however this approach required weakening our assumptions, resulting in a less robust formalization. We eventually simplified the termination proof by relying entirely on the static `rank` property. Since ranks are strictly monotonic and bounded by the heap size, the metric $\mu =$ `size` $-$ `rank` provided a stable, strictly decreasing measure that did not depend on the mutable shape of the tree. This simple insight facilitated the proof of termination without requiring complex graph traversal lemmas.

Future Work. A current limitation of our implementation is that its reliance on functional lists cause it to be inefficient for otherwise constant operations such as index-based accesses, and implies a linear time complexity for all operations that modify it, as the data must be entirely copied. An evolution of our work could include verifying a mutable version of our Union-Find implementation, which would allow a constant time complexity for the **Link** and **Make** if `IArray` are used. Using `IArray` would require minimal changes as the underlying implementation depends on `List`.

6 Conclusion

In this project, we successfully implemented and verified a Disjoint-Set Forest data structure in Stainless. By adapting the Separation Logic formalism of Charguéraud and Pottier into a first-order functional model, we demonstrated that complex, pointer-based algorithms can be verified in a pure functional setting.

Our main achievements include:

1. Replacing the complex "Memory Predicate" of separation logic with a flat-`List` model that is naturally supported by SMT solvers.
2. Providing a machine-checked proof that the operations of the Union-Find structure terminate, relying solely on rank monotonicity. Implicitly, this means that the graph is acyclic.
3. Developing a reusable library of `List` lemmas that solves the "frame problem," allowing us to verify updates locally without invalidating global invariants.

This project shows how Stainless performs well with structural verification. By explicitly representing the memory heap as a `List`, we achieved an almost fully verified, functional implementation of a fundamental data structure.

References

[1] Arthur Charguéraud and François Pottier. "Machine-Checked Verification of the Correctness and Amortized Complexity of an Efficient Union-Find Implementation". In: *6th International Conference on Interactive Theorem Proving (ITP)*. Nanjing, China, Aug. 2015. DOI: 10.1007/978-3-319-22102-1_9. URL: https://inria.hal.science/hal-01245872.

[2] Google. *Gemini 3*. https://gemini.google.com/. Large language model. 2026.

[3] EPFL LARA. *Stainless*. Dec. 2025. URL: https://epfl-lara.github.io/stainless/.

[4] OpenAI. *ChatGPT (GPT-5.2)*. https://chat.openai.com/. Large language model. 2026.

A Additional List Specifications

The following are lemmas used to prove many properties and invariants on the heap list. They have all been accepted by Stainless and are available in our repository at `src/MoreListSpecs.scala`.

A.1 Append lemmas

1. `appendToNil`: For all lists l and elements v, if $l = []$, then

$$v \in (l :+ v) \quad \text{and} \quad |l :+ v| = 1.$$

2. `snocNil`: For all elements v,

$$[] :+ v = [v].$$

3. `singletonList`: For all lists l and elements v, if $|l| = 1$ and $v \in l$, then

$$l = [v].$$

4. `appendedElementIsAtIndexOldSize`: For all lists l and elements e,

$$(l :+ e)(|l|) = e.$$

5. `snocForallAppend`: For all lists l, elements e, and predicates p, if

$$(\forall x \in l,\ p(x)) \wedge p(e),$$

then

$$\forall x \in (l :+ e),\ p(x).$$

6. `appendPreservesIndices`: For all lists l, elements n, and indices i with $0 \leq i < |l|$,

$$(l :+ n)(i) = l(i).$$

7. `appendFilterSizeDecreases`: For all lists l, elements n, and predicates p, if $p(n)$, then

$$|(l :+ n).\text{filter}(p)| = |l.\text{filter}(p)| + 1.$$

8. `appendFilterSizePreserved`: For all lists l, elements n, and predicates p, if $\neg p(n)$, then

$$|(l :+ n).\text{filter}(p)| = |l.\text{filter}(p)|.$$

A.2 Update lemmas

1. `forallUpdate`: For all lists l, indices a, elements e, and predicates p, if

$$0 \leq a < |l|, \quad (\forall x \in l,\ p(x)), \quad p(e),$$

then

$$\forall x \in l[a \leftarrow e],\ p(x).$$

2. `predicateIsPreservedOnNonUpdatedElements`: For all lists l, indices $a \neq i$, elements e, and predicates p, if

$$p(l(i)),$$

then

$$p(l[a \leftarrow e](i)).$$

3. `predicatePreservedOnNonUpdatedPair`: For all lists l, distinct indices $i_1 \neq i_2$, update index a with $a \neq i_1$ and $a \neq i_2$, and binary predicates p, if

$$p(l(i_1), l(i_2)),$$

then

$$p(l[a \leftarrow e](i_1),\ l[a \leftarrow e](i_2)).$$

4. `updatedFilterSizeIncreases`: For all lists l, indices a, elements e, and predicates p, if

$$p(e) \vee \neg p(l(a)),$$

then

$$|l[a \leftarrow e].\text{filter}(p)| \geq |l.\text{filter}(p)|.$$

5. `updatedFilterSizeIncreases2`: For all lists l, indices a, elements e, and predicates p, if

$$\neg p(l(a)) \wedge p(e),$$

then

$$|l[a \leftarrow e].\text{filter}(p)| = |l.\text{filter}(p)| + 1.$$

6. `updatePreservesIndices`: For all lists l, indices $a \neq i$, and elements e, if

$$0 \leq a, i < |l|,$$

then

$$l[a \leftarrow e](i) = l(i).$$

7. `updateOrderDoesNotMatter`: For all lists l, distinct indices $i_1 \neq i_2$, and elements e_1, e_2,

$$l[i_1 \leftarrow e_1][i_2 \leftarrow e_2] = l[i_2 \leftarrow e_2][i_1 \leftarrow e_1].$$

A.3 Slice lemmas

1. `sliceCons`: For all lists l and indices $from < until$,

$$l[from : until] = l(from) :: l[from + 1 : until].$$

2. `sliceAtIndex`: For all lists l_2, slices $l_1 = l_2[from : until]$, and indices $from \leq i < until$,

$$l_1(i - from) = l_2(i).$$

3. `sliceTail`: For all lists l_2 and slices $l_1 = l_2[from : until]$ with $|l_1| > 0$,

$$\text{tail}(l_1) = l_2[from + 1 : until].$$

4. `sliceZeroSize`: For all lists l,

$$l = l[0 : |l|].$$

A.4 Map lemmas

1. `mapDistributesOverAppend`: For all lists l, elements e, and functions f,

$$l.\text{map}(f) :+ f(e) = (l :+ e).\text{map}(f).$$

2. `mapContains`: For all lists l, functions f, and elements v, if $v \in l$, then

$$f(v) \in l.\text{map}(f).$$

3. `mapUpdate`: For all lists l, indices a, elements e, and functions f, if

$$f(l(a)) = f(e),$$

then

$$l[a \leftarrow e].\text{map}(f) = l.\text{map}(f).$$

4. `mapAtIndex`: For all lists l, indices i, and functions f,

$$l.\text{map}(f)(i) = f(l(i)).$$

A.5 Range lemmas

1. `rangeAppend`: For all integers $start \leq until$,

$$\text{range}(start, until) :+ until = \text{range}(start, until + 1).$$

2. `rangeAtIndexPlusStartIsIndexPlusStart`: For all integers $start$, $until$, and indices i with $start + i < until$,

$$\text{range}(start, until)(i) = start + i.$$

A.6 Filter lemmas

1. `weakenBoundOnListSize`: For all lists l, predicates p, and integers x, if

$$x \leq |l.\text{filter}(p)|,$$

then

$$x \leq |l|.$$